

AFRILANG Stdlib

std/c/cryptnonce

Français. Bibliothèque AFRILANG 'std/c/cryptnonce' (niveau complex, catégorie complex). Elle expose 8 fonction(s) : noncLen, noncUpper, noncLower, noncTrim, noncSplit, noncJoin, noncReplace, hash32.

```
'import "std/c/cryptnonce"' · 'use cryptnonce'
```

```
- 'noncLen(s text) → number' - 'noncUpper(s text) → text' - 'noncLower(s text) → text' - 'noncTrim(s text) → text' - 'noncSplit(s text, delim text) → list text' - 'noncJoin(parts list text, delim text) → text' - 'noncReplace(s text, from text, to text) → text' - 'hash32(s text) → number'
```

English.

AFRILANG library 'std/c/cryptnonce' (tier complex, category complex). Exports 8 function(s): noncLen, noncUpper, noncLower, noncTrim, noncSplit, noncJoin, noncReplace, hash32.

```
'import "std/c/cryptnonce"' · 'use cryptnonce'
```

```
- 'noncLen(s text) → number' - 'noncUpper(s text) → text' - 'noncLower(s text) → text' - 'noncTrim(s text) → text' - 'noncSplit(s text, delim text) → list text' - 'noncJoin(parts list text, delim text) → text' - 'noncReplace(s text, from text, to text) → text' - 'hash32(s text) → number'
```

Catégorie / Category Modules complex (c/)

Niveau / Tier complex

Fonctions / Functions 8

June 16, 2026

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/cryptnonce' (niveau complex, catégorie complex). Elle expose 8 fonction(s) : noncLen, noncUpper, noncLower, noncTrim, noncSplit, noncJoin, noncReplace, hash32.

```
'import "std/c/cryptnonce"' · 'use cryptnonce'
```

```
- `noncLen(s text) → number`
- `noncUpper(s text) → text`
- `noncLower(s text) → text`
- `noncTrim(s text) → text`
- `noncSplit(s text, delim text) → list text`
- `noncJoin(parts list text, delim text) → text`
- `noncReplace(s text, from text, to text) → text`
- `hash32(s text) → number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/cryptnonce"
use cryptnonce
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- noncLen(s text) → number•
noncUpper(s text) → text•
noncLower(s text) → text•
noncTrim(s text) → text•
noncSplit(s text, delim text) → list•
noncJoin(parts list text, delim text) → text•
noncReplace(s text, from text, to text) → text•
hash32(s text) → number

4. Exemples d'utilisation

```
import "std/c/cryptnonce"
use cryptnonce
```

```
create result = noncLen(/* args */)
say result
```

```
create result = noncUpper(/* args */)
say result
```

```
create result = noncLower(/* args */)
say result
```

```
create result = noncTrim(/* args */)
say result
```

```
create result = noncSplit(/* args */)
say result
```

```
create result = noncJoin(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/cryptnonce"`` en tête de fichier.
- Lancez ``afriLang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module cryptnonce

```
function noncLen(s text) returns number
end
```

```
function noncUpper(s text) returns text
end
```

```
function noncLower(s text) returns text
end
```

```
function noncTrim(s text) returns text
end
```

```
function noncSplit(s text, delim text) returns list text
end
```

```
function noncJoin(parts list text, delim text) returns text
end
```

```
function noncReplace(s text, from text, to text) returns text
end
```

```
function hash32(s text) returns number
end
end
```

English

1. Overview

AFRILANG library 'std/c/cryptnonce' (tier complex, category complex). Exports 8 function(s): noncLen, noncUpper, noncLower, noncTrim, noncSplit, noncJoin, noncReplace, hash32.

```
'import "std/c/cryptnonce"' · 'use cryptnonce'
```

```
- `noncLen(s text) → number`
- `noncUpper(s text) → text`
- `noncLower(s text) → text`
- `noncTrim(s text) → text`
- `noncSplit(s text, delim text) → list text`
- `noncJoin(parts list text, delim text) → text`
- `noncReplace(s text, from text, to text) → text`
- `hash32(s text) → number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/cryptnonce"
use cryptnonce
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- noncLen(s text) → number•
noncUpper(s text) → text•
noncLower(s text) → text•
noncTrim(s text) → text•
noncSplit(s text, delim text) → list•
noncJoin(parts list text, delim text) → text•
noncReplace(s text, from text, to text) → text•
hash32(s text) → number

4. Usage examples

```
import "std/c/cryptnonce"
use cryptnonce
```

```
create result = noncLen(/* args */)
say result
```

```
create result = noncUpper(/* args */)
say result
```

```
create result = noncLower(/* args */)
say result
```

```
create result = noncTrim(/* args */)
say result
```

```
create result = noncSplit(/* args */)
say result
```

```
create result = noncJoin(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/cryptnonce"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module cryptnonce

```
function noncLen(s text) returns number
end
```

```
function noncUpper(s text) returns text
end
```

```
function noncLower(s text) returns text
end
```

```
function noncTrim(s text) returns text
end
```

```
function noncSplit(s text, delim text) returns list text
end
```

```
function noncJoin(parts list text, delim text) returns text
end
```

```
function noncReplace(s text, from text, to text) returns text
end
```

```
function hash32(s text) returns number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/cryptnonce"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/cryptnonce/>

Source (excerpt)

```
module cryptnonce

function noncLen(s text) returns number
end

function noncUpper(s text) returns text
end

function noncLower(s text) returns text
end

function noncTrim(s text) returns text
end

function noncSplit(s text, delim text) returns list text
end

function noncJoin(parts list text, delim text) returns text
end

function noncReplace(s text, from text, to text) returns text
end

function hash32(s text) returns number
end
end
```